

C#.NET



Multithreading

Part-6

Mr. Bangar Raju

 **NARESHO**
technologies

Understanding Thread Priorities in C# .NET Multithreading

By Albert Harmon

Published April 14, 2017

🕒 3 min read

[Technology](#)



TLDR This blog post explores thread priorities in C# .NET multithreading, explaining how to set and utilize different priority levels to optimize CPU resource consumption for threads performing varying workloads.

In the realm of multithreading, managing how threads execute and share CPU resources is crucial for optimizing performance. This blog post delves into thread priorities in C# .NET, explaining their significance and demonstrating how to implement them effectively.

What are Thread Priorities?

Thread priorities determine the order in which threads are scheduled for execution by the operating system. In a multithreaded application, multiple threads may be executing simultaneously, and thread priorities help manage how CPU resources are allocated among them.

Default Behavior

By default, all threads in a C# .NET application run at a normal priority level. This means that if you have several threads, they will utilize CPU resources equally, without any variation in execution order.

Thread Priority Levels

C# .NET defines five different priority levels for threads, which are part of the `ThreadPriority`` enumeration:

1. **Lowest** - The least priority level.
2. **Below Normal** - Slightly higher than the lowest.
3. **Normal** - The default priority level.

4. **Above Normal** - Higher than normal.
5. **Highest** - The highest priority level.

These priority levels allow developers to specify which threads should receive more CPU resources based on their workload requirements.

Practical Implementation of Thread Priorities

To illustrate how thread priorities work, let's consider a practical example. We will create two threads that perform different tasks, and we will set their priorities accordingly.

Example Code

```
using System;
using System.Threading;

class ThreadPriorityExample
{
    static long count1 = 0;
    static long count2 = 0;

    public static void IncrementCount1()
    {
        while (true)
        {
```

```
        count1++;
    }
}

public static void IncrementCount2()
{
    while (true)
    {
        count2++;
    }
}

static void Main()
{
    Thread t1 = new Thread(IncrementCount1);
    Thread t2 = new Thread(IncrementCount2);

    // Set thread priorities
    t1.Priority = ThreadPriority.Lowest;
    t2.Priority = ThreadPriority.Highest;

    // Start threads
    t1.Start();
    t2.Start();

    // Let the main thread sleep for 10 seconds
    Console.WriteLine("Main thread going to sleep...");
    Thread.Sleep(10000);
    Console.WriteLine("Main thread woke up.");

    // Abort threads
```

```
t1.Abort();
t2.Abort();

// Display results
Console.WriteLine($"Count1: {count1}, Count2: {count2}
}
}
```

Explanation of the Code

1. **Thread Creation:** We create two threads, `t1` and `t2`, each responsible for incrementing a separate counter indefinitely.
2. **Setting Priorities:** We set `t1` to the lowest priority and `t2` to the highest priority. This means `t2` will receive more CPU resources compared to `t1`.
3. **Execution:** After starting both threads, the main thread sleeps for 10 seconds, allowing the other threads to run.
4. **Aborting Threads:** After waking up, the main thread aborts both threads and prints the final values of `count1` and `count2`.

Observations

When running the above code, you will notice that `count2` (the thread with the highest priority) typically has a higher value than `count1`. This is because `t2` is consuming more CPU resources due

to its higher priority, allowing it to increment its counter more frequently.

Conclusion

Thread priorities in C# .NET provide a powerful mechanism for optimizing the performance of multithreaded applications. By setting appropriate priorities based on the workload of each thread, developers can ensure that critical tasks receive the necessary CPU resources, leading to more efficient execution. Understanding and utilizing thread priorities is essential for anyone looking to develop robust multithreaded applications in C#.